

1. What is Airflow?

Apache Airflow is an open-source workflow orchestration platform used to design, schedule, execute, and monitor data pipelines. It was originally developed by Airbnb and later became a top-level project of the Apache Software Foundation. Airflow allows organizations to automate complex workflows by defining them as code using Python. Instead of manually running scripts and jobs, users can schedule workflows to run automatically at specific times or based on dependencies.

Key Features

- Workflow automation
- Task scheduling
- Dependency management
- Monitoring and logging
- Retry and failure handling
- Scalability across multiple machines

Why Apache Airflow is Needed

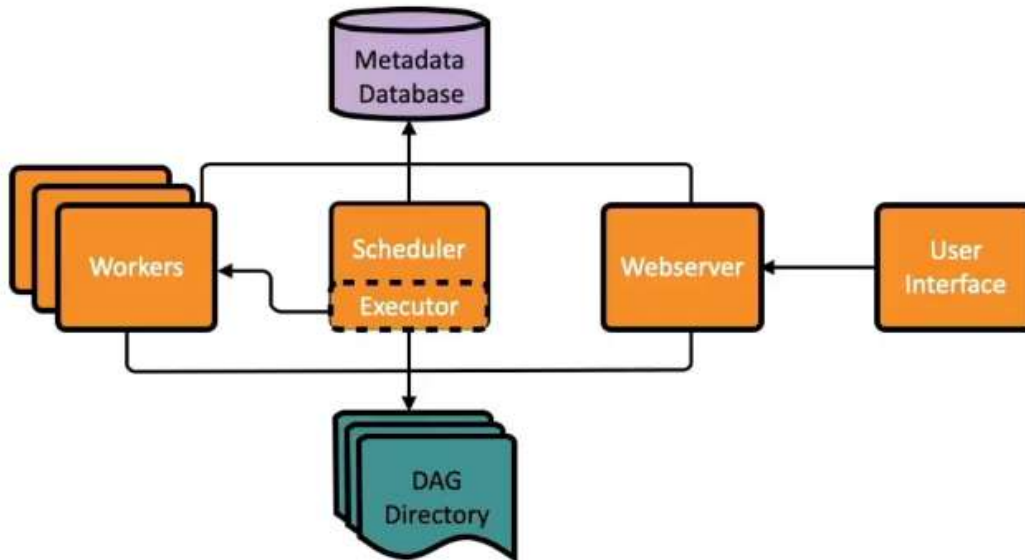
Modern data systems involve multiple steps such as extracting data from source systems, transforming the data, loading it into databases, generating reports, and notifying users. Managing these steps manually is inefficient and prone to errors. Apache Airflow solves this problem by automating the entire workflow and ensuring tasks are executed in the correct sequence.

Benefits

- Eliminates manual execution
- Ensures task dependencies are maintained
- Provides centralized monitoring
- Supports automatic retries
- Reduces operational effort

Airflow Architecture

Apache Airflow consists of several components that work together to execute workflows. The Scheduler determines when tasks should run, the Executor executes the tasks, the Metadata Database stores workflow information, and the Web Server provides a user interface for monitoring and management.



Main Components

1. Directed Acyclic Graph (DAG)

A DAG (Directed Acyclic Graph) is the heart of Apache Airflow. It represents a workflow and defines the sequence in which tasks should be executed. Since a DAG is acyclic, it does not allow circular dependencies or infinite loops.

Characteristics of DAGs:

- Define workflow structure
- Manage task dependencies
- Prevent circular execution paths
- Written using Python code

2. Tasks

A task is the smallest unit of work within a DAG. Each task performs a specific operation such as reading data, running a query, processing files, or sending notifications.

Examples of tasks include:

- Reading a CSV file
- Running a SQL query
- Executing a Python script
- Sending an email alert
- Launching a Spark job

By dividing workflows into tasks, Airflow makes execution and monitoring easier.

3. Operators

Operators define what action a task performs. Airflow provides many built-in operators that simplify workflow development.

Common Operators

Operator	Purpose
PythonOperator	Executes Python functions
BashOperator	Executes shell commands
EmailOperator	Sends email notifications
SQL Operator	Runs SQL queries
DummyOperator	Placeholder task

Operators help integrate Airflow with databases, cloud platforms, APIs, and other tools.

4. Scheduler

The Scheduler is responsible for monitoring DAGs and triggering them according to their schedule. It continuously checks workflow definitions and creates task instances when execution time arrives.

Responsibilities

- Monitors DAG files
- Creates task instances
- Manages execution timing
- Enforces dependencies

Without the Scheduler, Airflow would not be able to automate workflows.

5. Executor

The Executor determines how and where tasks are executed. Different executors are available depending on workload size and infrastructure requirements.

Types of Executors

- ❖ **Sequential Executor** – Runs one task at a time.
- ❖ **Local Executor** – Runs multiple tasks on a single machine.
- ❖ **Celery Executor** – Distributes tasks across multiple worker nodes.
- ❖ **Kubernetes Executor** – Runs tasks inside Kubernetes pods.

Production environments typically use Celery or Kubernetes Executors for scalability.

6. Metadata Database

The Metadata Database acts as Airflow's central storage system. It stores information about DAGs, task execution history, schedules, logs, and user configurations.

Stores Information

- DAG definitions
- Task status
- Execution history
- User accounts
- Workflow schedules
- Logs

Common databases used are PostgreSQL and MySQL.

7. Web User Interface

Airflow provides a web-based interface that allows users to monitor workflows visually. Users can view DAG execution status, inspect logs, restart failed tasks, and trigger workflows manually.

Features of Airflow UI

- DAG visualization
- Task monitoring
- Log viewing
- Manual triggering
- Failure analysis
- Workflow history tracking

The UI greatly simplifies workflow management and troubleshooting.

8. Scheduling

Scheduling is one of Airflow's most important features. Workflows can be configured to run at fixed intervals or according to custom cron expressions.

Common Schedules

- @once
- @hourly
- @daily
- @weekly
- @monthly

Retry and Failure Handling

In real-world environments, failures can occur because of network issues, unavailable services, or temporary system problems. Airflow includes built-in retry mechanisms that automatically rerun failed tasks.

Airflow in Data Engineering

Apache Airflow is one of the most widely used orchestration tools in Data Engineering. It helps manage complex pipelines that involve multiple technologies and systems.

Common Use Cases

- ❖ ETL and ELT Pipelines
- ❖ Data Warehouse Loading
- ❖ Azure Databricks Jobs
- ❖ PySpark Processing
- ❖ Machine Learning Pipelines
- ❖ Data Lake Automation
- ❖ Report Generation

2. Why Airflow instead of using Azure Data Factory.

Airflow:

Apache Airflow is preferred when organizations require highly customizable workflows. Since workflows are written entirely in Python, developers can create dynamic pipelines, implement complex business logic, integrate with multiple

cloud providers, and orchestrate various technologies from a single platform. Airflow is especially popular in Data Engineering teams where Python is already widely used. It provides complete control over workflow execution and can orchestrate jobs running on Azure, AWS, Google Cloud, Databricks, Spark, Hadoop, APIs, and databases.

Advantages of Airflow

- Fully code-based (Python)
- Supports complex workflows
- Strong dependency management
- Multi-cloud compatibility
- Easy integration with Spark and Databricks
- Open-source with no vendor lock-in
- Dynamic DAG generation

When Airflow is Better

Airflow is a better choice when:

- ❖ Complex workflow dependencies exist.
- ❖ Heavy Python-based processing is required.
- ❖ Multi-cloud environments are involved.
- ❖ Custom business logic is extensive.
- ❖ Teams want complete control over orchestration.
- ❖ Open-source tools are preferred.

Azure Data Factory:

Azure Data Factory is preferred when organizations already use Azure services extensively. It provides a visual interface where pipelines can be created with minimal coding. Microsoft manages the infrastructure, scaling, monitoring, and maintenance, allowing teams to focus on business requirements rather than platform administration.

Advantages of Azure Data Factory

- No server management
- Easy drag-and-drop development
- Built-in Azure integration
- Fast development cycle
- Enterprise security and governance
- Automatic scaling

When Azure Data Factory is Better

ADF is a better choice when:

- ❖ The organization uses Azure as its primary platform.
- ❖ Pipelines are mostly ETL/ELT based.
- ❖ Low-code development is preferred.
- ❖ Infrastructure management should be avoided.
- ❖ Business users participate in pipeline development.

Airflow vs Azure Data Factory

Feature	Apache Airflow	Azure Data Factory
Type	Open-source orchestration tool	Cloud ETL/ELT service
Development	Python code	Drag-and-drop UI
Hosting	Self-managed	Fully managed by Azure
Flexibility	Very high	Moderate
Cost	Infrastructure cost only	Pay-per-use
Multi-cloud support	Yes	Primarily Azure
Custom Logic	Excellent	Limited compared to Python
Monitoring	Good	Excellent Azure integration

3. What is DAG(Directed Acyclic Graph)?

A **DAG (Directed Acyclic Graph)** is the fundamental concept in Apache Airflow. Every workflow created in Airflow is represented as a DAG. It acts as a blueprint that defines the tasks to be executed, the order in which they should run, and the dependencies between them. Airflow uses DAGs to understand how a workflow should execute from start to finish.

In simple terms, a DAG is a collection of tasks organized in a logical sequence where each task depends on one or more previous tasks. Airflow reads the DAG definition and automatically executes the tasks according to the specified dependencies.

Understanding the Term DAG

The term DAG consists of three words:

1. Directed

The word Directed means that every task has a defined direction of execution. One task leads to another task in a specific sequence.

Key Points

- Tasks execute in a specific order.
- Dependencies determine the flow.
- Airflow follows the defined direction during execution.

2. Acyclic

The word Acyclic means that the workflow cannot contain loops or circular dependencies.

Valid DAG:

Task A → Task B → Task C

Key Points

- No circular dependencies.
- No infinite loops.
- Each workflow has a clear beginning and end.

3. Graph

A graph is a collection of connected nodes.

In Airflow:

- Nodes represent Tasks.
- Connections represent Dependencies.

Key Points

- Tasks are represented as nodes.
- Dependencies are represented as edges.
- Multiple tasks can be connected together.

Components of a DAG

A DAG consists of several important components.

1. DAG ID

Every DAG must have a unique identifier called the DAG ID.

Example:

```
dag_id="employee_attendance_pipeline"
```

The DAG ID helps Airflow identify and manage workflows.

Purpose

- Uniquely identifies a workflow.
- Used for monitoring and scheduling.
- Displayed in the Airflow UI.

2. Tasks

Tasks are the individual units of work within a DAG.

Examples:

- Read CSV file
- Execute SQL query
- Run PySpark job

- Execute Databricks notebook
- Send email notification

A DAG may contain one task or hundreds of tasks.

3. Dependencies

Dependencies define the execution order of tasks.

Example:

```
task1 >> task2 >> task3
```

Benefits

- Ensures proper execution order.
- Prevents premature task execution.
- Maintains workflow consistency.

4. Scheduling

Scheduling determines when the DAG should run.

Airflow supports multiple scheduling options.

Example:

```
schedule="@daily"
```

This DAG will execute every day automatically.

How DAG Execution Works

When a DAG is triggered, Airflow follows a sequence of steps.

1. Scheduler reads the DAG.
2. Dependencies are analyzed.
3. Tasks ready for execution are identified.
4. Executor runs the tasks.
5. Results are stored in the metadata database.
6. Logs are generated for monitoring.

Advantages of DAGs

- ❖ Clear workflow structure.
- ❖ Easy dependency management.
- ❖ Supports parallel execution.
- ❖ Prevents circular dependencies.
- ❖ Improves reliability.
- ❖ Enables automation.
- ❖ Simplifies monitoring and troubleshooting.
- ❖ Scales for large data pipelines

4. Execute all the KillerCoda Steps.

Step 1: Open KillerCoda Environment

1. Open the KillerCoda Airflow lab.
2. Click Start Scenario.
3. Wait for the environment to initialize.
4. Open the terminal provided by KillerCoda.

Verify Docker is running:

```
docker --version
```

Verify Airflow image:

```
docker images
```

Step 2: Create Airflow Project Directory

```
mkdir airflow-project
```

```
cd airflow-project
```

Create required folders:

```
mkdir dags logs plugins
```

Verify:

```
ls
```

Output:

```
dags
```

```
logs
```

```
plugins
```

Step 3: Download Airflow Docker Compose File

```
curl -Lfo 'https://airflow.apache.org/docs/apache-airflow/stable/docker-compose.yaml'
```

Check file:

```
ls
```

Output:

```
docker-compose.yaml
```

```
dags
```

```
logs
```

```
plugins
```

Step 4: Create Airflow Environment Variable

```
echo -e "AIRFLOW_UID=$(id -u)" > .env
```

Verify:

```
cat .env
```

Step 5: Initialize Airflow Database

Run:

```
docker compose up airflow-init
```

This step:

- Creates Metadata Database
- Creates Airflow tables
- Creates Admin User

Successful message:

airflow-init exited with code 0

Step 6: Start Airflow Services**Run:**

docker compose up -d

Check containers:

docker ps

Should see:

airflow-webserver

airflow-scheduler

airflow-worker

postgres

redis

Step 7: Open Airflow UI

Get KillerCoda web preview.

Usually:

Port 8080

Open:

<http://localhost:8080>

Login:

Username: admin

Password: "random one"

Step 8: Verify Airflow is Running

Check:

docker ps

All containers should show:

STATUS: Up

Step 9: Create Your First DAG

Move into DAG folder:

```
cd dags
```

Create DAG file:

```
nano first_dag.py
```

Paste: Dag File need to execute.

Save: CTRL + D and ENTER

Step 10: Verify DAG is Loaded

Return to Airflow UI.

Go to: DAGs

Should see: first_dag

If not:

```
docker compose restart
```

Wait 1 minute and refresh.

Step 11: Enable the DAG

In Airflow UI:

- Locate first_dag
- Turn ON the toggle switch

Status:

Paused → Active

Step 12: Trigger the DAG

Click:

first_dag

Then:

Trigger DAG

Airflow creates a DAG Run.

Step 13: View Graph

Open: Graph View

View: Extract -> Transform -> Load

This represents the DAG execution flow.

Step 14: Monitor Execution

Go to: Grid View

Task colors:

Color	Meaning
Green	Success
Red	Failed
Yellow	Running
Gray	Not Started

Step 15: View Task Logs

Click: extract

Then: Logs

Output:

Extracting Data

For Transform:

Transforming Data

For Load:

Loading Data

Step 16: Check DAG Execution from CLI

List DAGs:

```
docker exec -it airflow-airflow-scheduler-1 airflow dags list
```

List DAG runs:

```
docker exec -it airflow-airflow-scheduler-1 airflow dags list-runs -d first_dag
```

Output:

```
dag_id    state
```

```
first_dag success
```

Step 17: Trigger DAG Using CLI

Instead of UI:

```
docker exec -it airflow-airflow-scheduler-1 airflow dags trigger first_dag
```

Check status:

```
docker exec -it airflow-airflow-scheduler-1 airflow dags list-runs -d first_dag
```

Step 18: Verify Successful Execution

A successful execution will show:

```
extract_task    success
```

```
transform_task  success
```

```
load_task       success
```

And in Airflow UI:

DAG Status = Success

These are the complete Airflow/KillerCoda steps from environment setup through successful DAG execution.

5. Execute your first DAG.

To execute my first DAG file in Apache Airflow by following these steps one-by-one:

Step 1: Open KillerCoda Environment

1. Open the KillerCoda Airflow in any browser.
2. Click the Playgrounds and Ubuntu 24.04

3. Now make a sign in.
4. Click Start Scenario.
5. Wait for the environment to initialize.
6. Open the terminal provided by KillerCoda.

Step 2: Run Airflow Container

```
docker run -d -p 8080:8080 --name airflow apache/airflow standalone
```

Step 3: Verify Container is Running

```
docker ps
```

Step 4: Check Airflow Logs

```
docker logs airflow
```

Important Information:

Take the User credentials of password and user name to log into the Airflow UI.

Step 5: Open an Airflow UI.

1. Click the Traffic/Ports at right side of the terminal.
2. Now Choose the 8080 port of the option showing.
3. Now use the user credentials to log in the UI.
4. Airflow UI will be viewed.

Step 6: Enter the Airflow Container

```
docker exec -it airflow bash
```

Step 7: Navigate to DAG Folder

```
cd /opt/airflow/dags
```

Step 8: Create First DAG

1. Type: cat > first_dag.py
2. Now paste the first_dag.py file in it.
3. Save by CTRL + D.

Example of first_dag.py:

```
from airflow.sdk import DAG
```

```
from airflow.providers.standard.operators.python import PythonOperator
```

```

from datetime import datetime

def task1_function():
    print("Task 1 Started")

def task2_function():
    print("Task 2 Running")

def task3_function():
    print("Task 3 Completed")

with DAG(
    dag_id="first_training_dag",
    start_date=datetime(2025, 1, 1),
    schedule="@daily",
    catchup=False,
    tags=["training"]
) as dag:
    task1 = PythonOperator(
        task_id="task1",
        python_callable=task1_function )
    task2 = PythonOperator(
        task_id="task2",
        python_callable=task2_function )
    task3 = PythonOperator(
        task_id="task3",
        python_callable=task3_function )
    task1 >> task2 >> task3

```

Step 9: Verify DAG File Exists

```
ls
```

Step 10: Check Airflow DAG Directory

airflow config get-value core dags_folder

Step 11: Check DAG Import Errors

airflow dags list-import-errors

Step 12: Reserialize DAGs

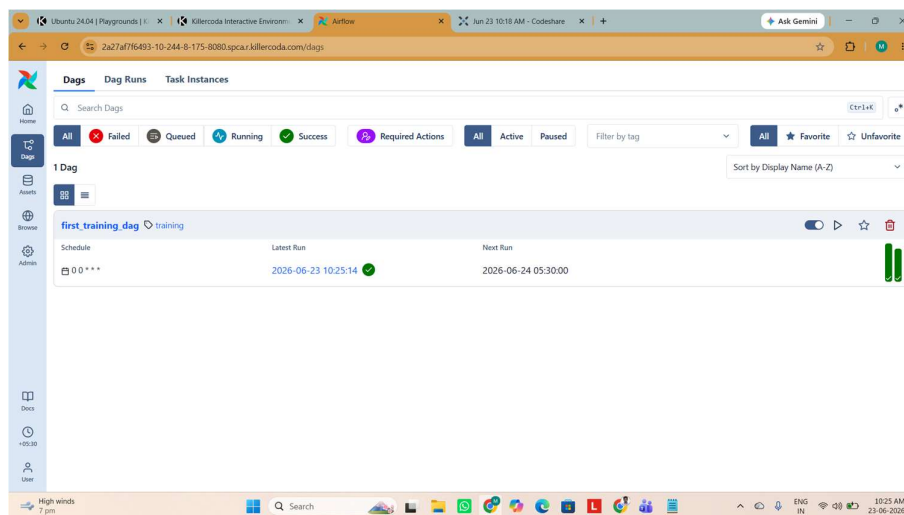
airflow dags reserialize

Step 13: List Available DAGs

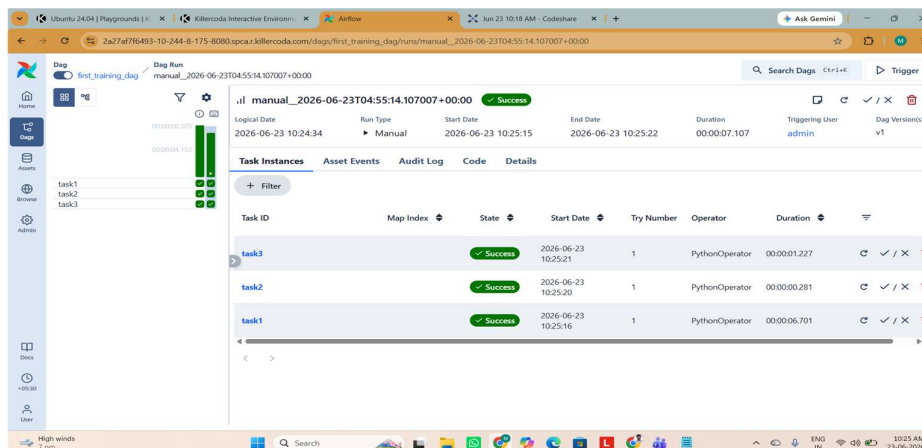
airflow dags list

Step 14: Final Output at Airflow UI.

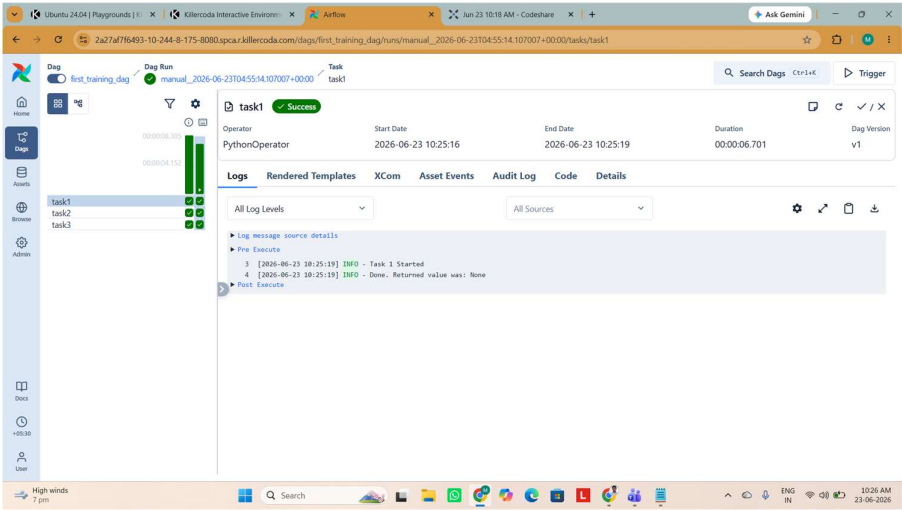
first_day.py:



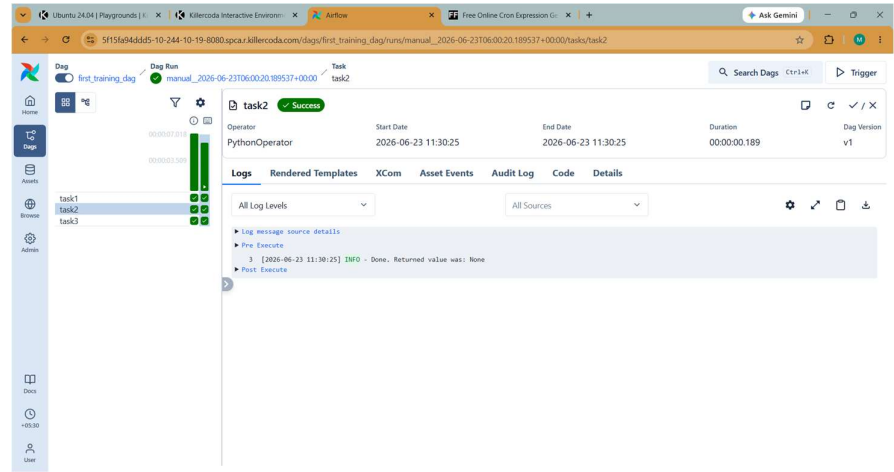
first_training_dag:



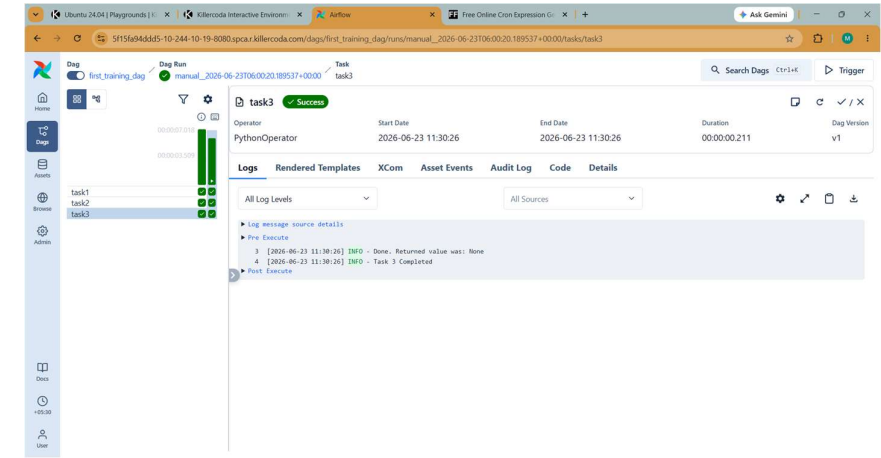
Task – 1:



Task – 2:



Task – 3:



By following these steps successfully created & executed the first dags.